Alma Mater Studiorum · Università di Bologna

CORSO DI LAUREA IN INGEGNERIA E SCIENZE INFORMATICHE

Smart Tank monitoring System

Relazione per il corso di Sistemi Embedded e IoT

Studente:
Riem Boukhama
0001129990

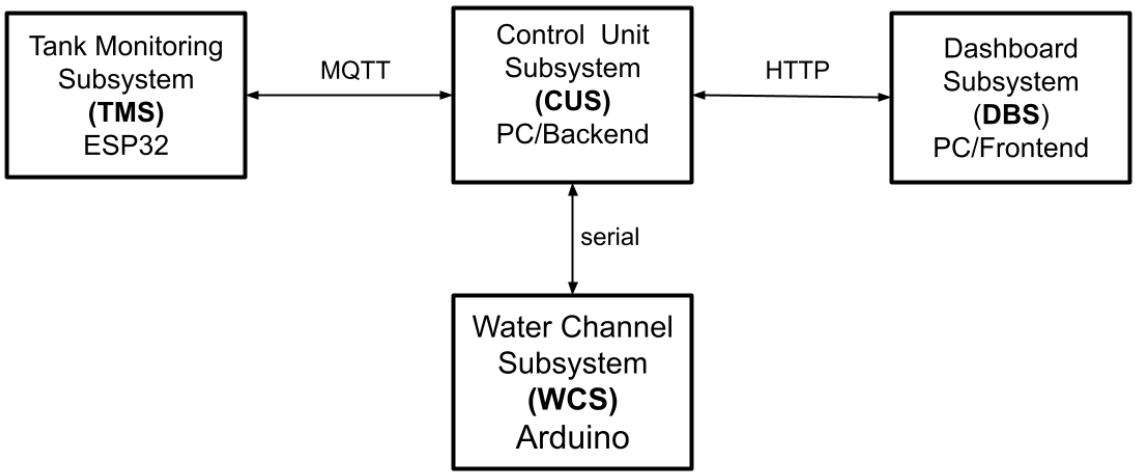
Anno Accademico 2025/2026

1 Introduzione e Architettura del Sistema

Il sistema **Smart Tank** è un'architettura progettata per il monitoraggio del livello d'acqua in un serbatoio e il controllo dell'apertura di una valvola di scarico, al fine di gestirne il riempimento.

L'architettura del progetto è suddivisa in quattro sottosistemi principali:

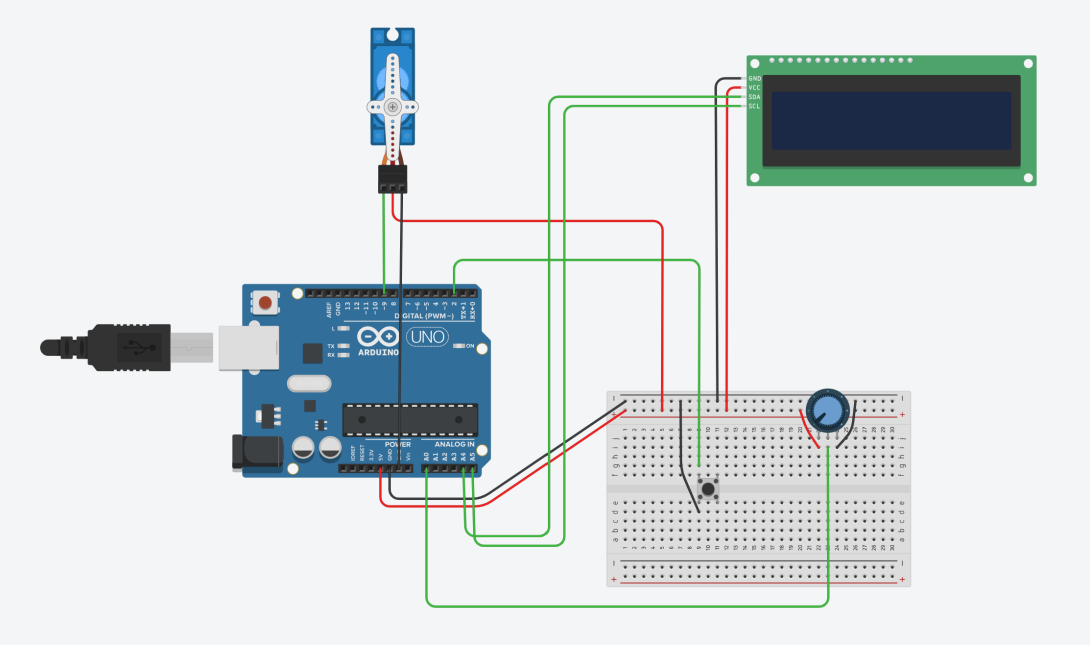
- **TMS (Tank Monitoring Subsystem):** Nodo basato su microcontrollore ESP32, incaricato del monitoraggio del livello d'acqua e della comunicazione con il CUS tramite MQTT.
- **WCS (Water Channel Subsystem):** Nodo basato su Arduino Uno, incaricato al controllo fisico dell'attuatore della valvola, alla gestione del display LCD e alla ricezione degli input fisici (pulsante e potenziometro).
- **CUS (Control Unit Subsystem):** Unità sviluppata in python ed eseguita su PC che funge da server di controllo. Implementa le regole operative del sistema, coordina la comunicazione tra Arduino e il server MQTT ed espone un server HTTP per la dashboard.
- **DBS (Dashboard Subsystem):** Interfaccia operatore web accessibile da browser. Fornisce un monitoraggio grafico in tempo reale del livello dell'acqua e permette l'invio di comandi al sistema.



3 Schema Hardware

Il sistema include due unità di controllo distinte disposte su aree separate della breadboard, operanti su Arduino Uno ed ESP32.

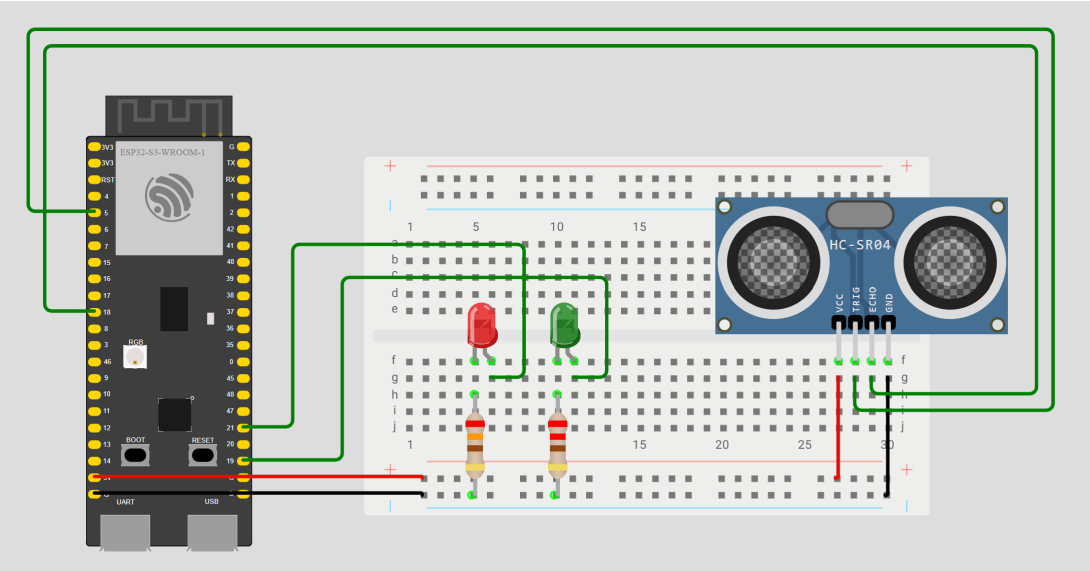
Nodo WCS (Arduino Uno R3)



Per Arduino Uno R3 sono stati usati i seguenti componenti:

- **Servomotore (HS-53)**.
- **Pulsante tattile**.
- **Potenziometro Analogico**
- **Display LCD I2C**

Nodo TMS (ESP32-S3)



Per ESP32-S3 sono stati usati i seguenti componenti:

- **Sensore Sonar (HC-SR04)**
- **LED Verde**
- **LED Rosso**

4 Architettura Software

Il firmware del Water Channel Subsystem (WCS) è stato sviluppato in linguaggio C++ secondo il principio della programmazione ad oggetti, adottando un'architettura a livelli..

Hardware Abstraction Layer (HAL)

Alla base dell'architettura si trova l'Hardware Abstraction Layer (HAL), il cui compito è astrarre gli elementi del circuito esponendo interfacce e metodi indipendenti dall'hardware.

Ogni dispositivo fisico del circuito è stato modellato in una classe dedicata:

- `Button`**:** modella il pulsante per il cambio della modalità. L'interfaccia astratta definisce il metodo (`isPressed()`), mentre l'implementazione (`ButtonImpl`) gestisce la configurazione del pin e la lettura del segnale analogico.
- `Potentiometer`**:** astrae il potenziometro per il controllo manuale della valvola. La classe `PotentiometerImpl` incapsula l'acquisizione del segnale analogico tramite `analogRead()`.
- `ServoMotor`**:** gestisce l'attuatore della valvola del serbatoio. La classe `ServoMotorImpl` converte una percentuale di apertura richiesta ($0-100\%$) nel corrispondente angolo ($0-90°$) inviato al servomotore.
- `Lcd`**:** astrae il display I2C. L'implementazione `LcdImpl` gestisce i comandi di inizializzazione, retroilluminazione e stampa dei messaggi a schermo tramite `printStatus()`.

Architettura ad Eventi

L'esecuzione del firmware è strutturata secondo il paradigma *event-triggered*. A differenza dei modelli time-triggered sincroni, nei quali il flusso operativo è gestito da un periodi di tempo prefissati, le transizioni tra uno stato e l'altro hanno luogo esclusivamente al verificarsi di eventi generati dal sistema stesso.

L'architettura software si basa su tre aspetti fondamentali:

1. Tipologia di evento

Gli eventi esterni ed interni vengono modellati sotto forma di istanze derivate dalla classe `Event`:

- **Eventi Hardware (`EV_BUTTON`):** generati tramite una Interrupt Service Routine (ISR) collegata al pin del pulsante.
- **Eventi di Comunicazione (`EV_SERIAL`):** generati quando una riga di comando inviata dal CUS viene estratta dal buffer.
- **Eventi Temporali (`EV_TIMER`):** generati a intervalli regolari di 50 ms tramite la funzione `millis()`, permettendo di rilevare periodicamente il valore del potenziometro.

2. Gestione eventi in entrata

Gli eventi verificati sono inseriti una coda circolare `EventQueue`, in questo modo si rende possibile il disaccoppiamento tra la fase di generazione dell'evento e la successiva risposta ed esecuzione delle azioni.

Nell'aggiornamento dei puntatori della queue, per evitare condizioni di conflitto causati dalla concorrenza della ISR (*interrupt service routine*), i metodi `enqueue()` e `dequeue()` gestiscono le manipolazioni della queue all'interno di sezioni critiche facendo uso delle funzioni `noInterrupts()` e `interrupts()`.

3. Elaborazione degli eventi

All'interno del superloop, l'*event loop* si occupa di estrarre gli eventi pendenti dalla queue e inoltrarli alla `FSM` per la loro esecuzione:

1. Ad ogni iterazione, il loop verifica se la coda contiene elementi tramite `!eventQueue.isEmpty()`. In caso positivo, estrae l'evento `ev` in testa con `dequeue()`.
2. Il puntatore all'evento viene inviato alla `FSM` invocando la funzione `fsm->handleEvent(ev)`. La FSM controlla il tipo di evento estratto (`getType()`), lo stato attivo in quel momento e transita allo stato opportuno.
3. Non appena la `FSM` ha terminato le proprie azioni, il controllo torna al loop che cancella l'oggetto tramite l'istruzione `delete ev` per liberare l'evento dalla ram.

5 Logiche Operative

Water Channel Subsystem (WCS)

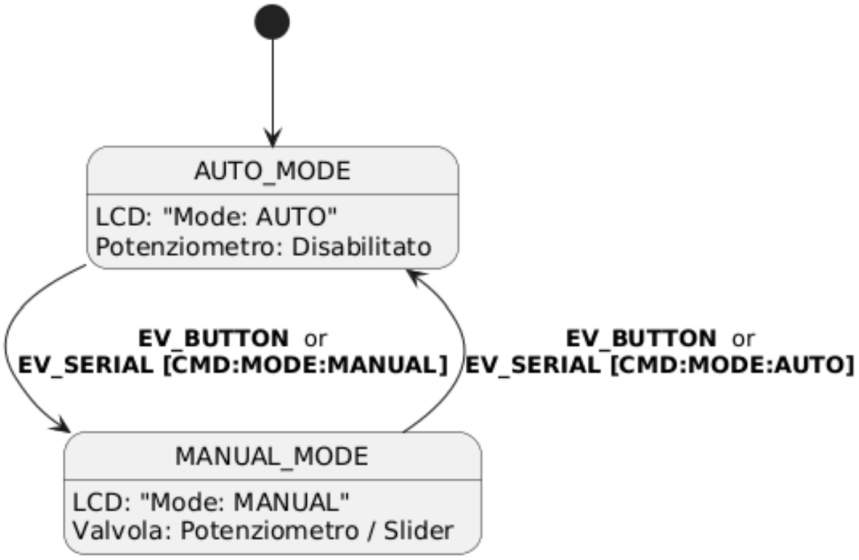


Figura 1: Diagramma degli stati del nodo WCS.

Stati Operativi

All'avvio (`setup()`), il WCS inizializza il display LCD, porta il servomotore a 0° (0%) ed entra nello stato iniziale `AUTO_MODE`. Gli stati possibili sono due:

- `AUTO_MODE`**:** La valvola è controllata in automatico dal CUS in base al livello dell'acqua, Sul display LCD viene stampato `Mode: AUTO`, insieme alla percentuale di apertura della valvola. Gli input ricevuti dal potenziometro vengono ignorati.
- `MANUAL_MODE`**:** L'apertura della valvola è gestita manualmente dell'operatore. Il display stampa `Mode: MANUAL`. L'operatore può regolare la valvola utilizzando il potenziometro fisico oppure utilizzando lo slider della dashboard.

Transizioni tra Stati

- **Da** `AUTO_MODE` **a** `MANUAL_MODE`**:** Si attiva alla pressione del pulsante tattile oppure all'invio del comando `CMD:MODE:MANUAL` dalla Dashboard.
- **Da** `MANUAL_MODE` **a** `AUTO_MODE`**:** Si attiva premendo nuovamente il pulsante (`EV_BUTTON`) o tramite il comando `CMD:MODE:AUTO`.

A ogni cambio di stato, il WCS aggiorna lo schermo LCD e invia al CUS la stringa `STATE:<MODE>,VALVE:<val>` con `sendStateToSerial()`, in modo da aggiornare in tempo reale lo stato sulla Dashboard.

Tank Monitoring Subsystem (TMS)

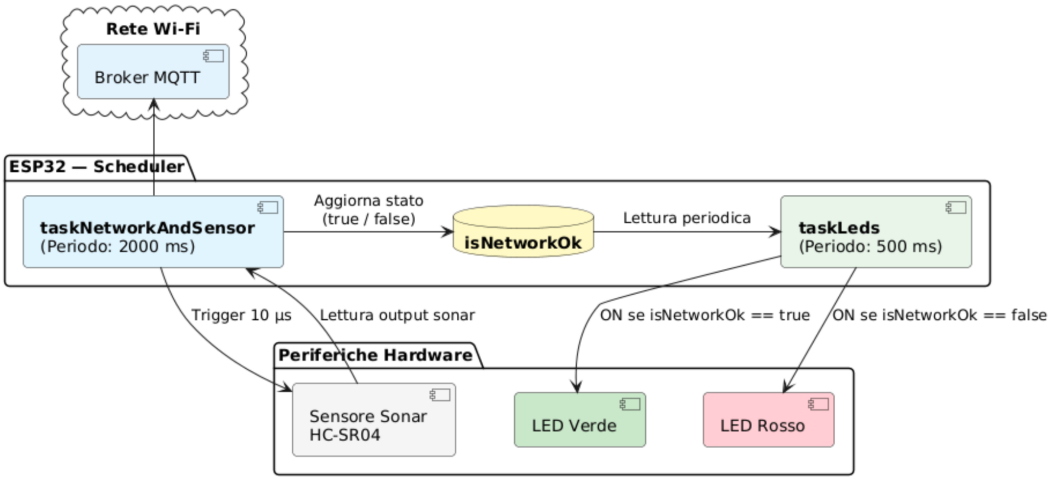


Figura 2: Architettura a task del nodo TMS.

Il lavoro dei task è suddiviso in questo modo:

- `taskNetworkAndSensor`: Si ripete ogni 2 secondi e si occupa di monitorare la rete e misurare il livello dell'acqua.
  - Controlla che il Wi-Fi e la connessione con il broker MQTT (`broker.hivemq.com`) siano attivi. Se avviene una disconnessione imposta la variabile `isNetworkOk` a `false` e attende un secondo prima di riprovare.
  - Quando si ristabilisce la connessione, imposta la variabile `isNetworkOk` a `true`.
  - Usa il sensore sonar per misurare il riempimento del tank.
  - Converte la misura in una percentuale da 0 a 100, considerando un'altezza del serbatoio di 100 cm.
  - Invia il valore ottenuto tramite protocollo MQTT sul topic `smart_tank/level`.
- `taskLeds`: Si ripete ogni 0.5 secondi e controlla i due LED.
  - Legge lo stato della variabile `isNetworkOk`.
  - Se è `true` (connesso), accende il LED verde sul pin 19 e spegne il LED rosso sul pin 21.
  - Se è `false` (disconnesso), spegne il LED verde e accende il LED rosso.

La suddivisione dell'applicazione in due task concorrenti separa il controllo della rete dalla gestione dei LED, garantendo l'esecuzione di entrambe le funzioni contemporaneamente. La comunicazione tra i due task è affidata alla variabile isNetworkOk, aggiornata dal task `taskNetworkAndSensor` e verificata dal task `taskLeds`.

Control Unit Subsystem (CUS)

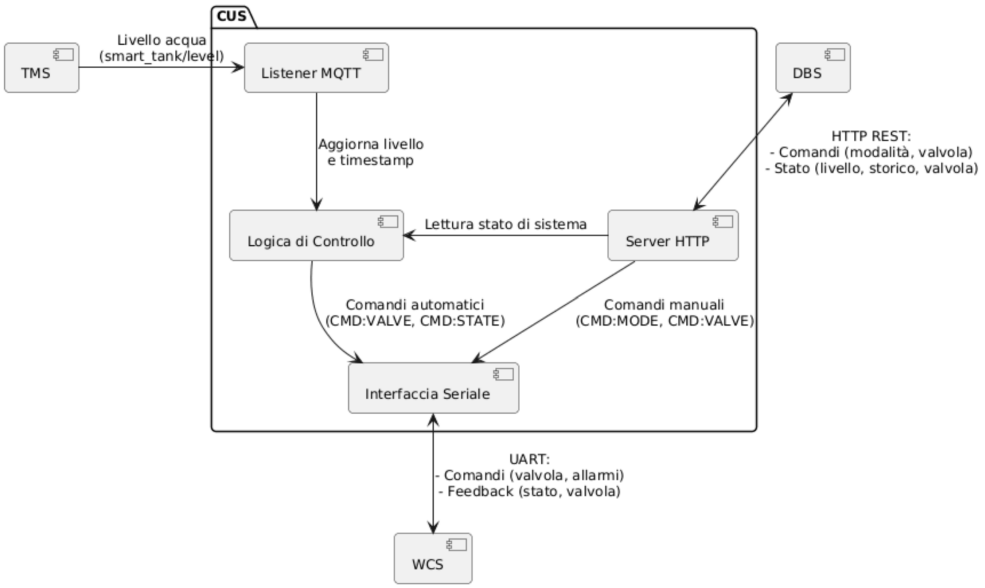


Figura 3: Architettura del CUS e flussi di comunicazione con i sottosistemi TMS, WCS e DBS.

Architettura Software

Per coordinare canali di I/O, il CUS adotta un'architettura basata su quattro thread:

- **Thread Web Server:** Espone un'interfaccia HTTP REST e mette a disposizione gli endpoint `/api/state` (GET) per la sincronizzazione dello stato del sistema con la Dashboard e `/api/command` (POST) per la ricezione dei comandi eseguiti dall'operatore.
- **Thread Client MQTT:** Mantiene una connessione con il broker (`broker.hivemq.com`) tramite il topic `smart_tank/level`. A ogni ricezione, converte il dato in percentuale, aggiorna il timestamp dell'ultimo pacchetto valido, memorizza il livello dell'acqua e lo inserisce all'interno di una lista che servirà al tracciamento del grafico del livello dell'acqua.
- **Thread di Comunicazione:** Esegue un ciclo di polling sulla porta UART. Effettua il parsing delle stringhe trasmesse dal WCS nel formato `STATE:<MODE>,VALVE:<val>`, aggiornando la modalità corrente e la percentuale di apertura della valvola.
- **Thread di Controllo:** Controlla periodicamente lo stato della rete e valuta i dati del TMS per regolare l'apertura della valvola quando si è in modalità `AUTO`.

Logica Operativa

Lo stato del sistema cambia in base a parametri e condizioni temporali ben precisi:

- **Stato della rete:** Il thread che controlla la rete calcola costantemente la differenza temporale tra l'istante attuale e l'istante dell'ultimo messaggio valido ricevuto dal TMS. Qualora l'intervallo temporale superi la soglia di timeout prefissata ($\Delta t > T_2$, con $T_2 = 10.0s$), lo stato del sistema passa a `UNCONNECTED` e viene trasmesso il comando `CMD:STATE:UNCONNECTED`

verso il WCS. Al ripristino della connessione entro T_2, lo stato torna a `NORMAL` inviando il messaggio `CMD:STATE:CONNECTED`.

- **Modifica apertura valvola:** La regolazione automatica della valvola è attiva esclusivamente quando lo stato del sistema è `NORMAL` e la modalità del WCS è `AUTO`:
  - **($L > L_2 = 80\%$):** Il CUS trasmette sulla porta seriale il comando `CMD:VALVE:100` per aprire completamente la valvola.
  - **($L_1 > L > L_2 = 50\%$):** Se il livello dell'acqua rimane sopra alla soglia (L_1) per un intervallo $t \geq T_1 = 5.0s$, la valvola viene aperta al 50%, inviando il comando `CMD:VALVE:50`.
  - **($L \leq L_1 = 0\%$):** Viene inviato il comando `CMD:VALVE:0` per chiudere completamente la valvola.

Gestione dei comandi manuali

Tramite le richieste POST sull'endpoint `/api/command`, il CUS gestisce le azioni ricevute dal DBS:

- Se la richiesta contiene un cambio della modalità (`mode`), il backend converte l'azione in `CMD:MODE:AUTO` o `CMD:MODE:MANUAL` e lo invia sul canale seriale al WCS.
- Se la richiesta include una regolazione dell'apertura della valvola (`valve`), il comando `CMD:VALVE:<val>` viene inviato al WCS esclusivamente se la modalità attuale del WCS risulta essere `MANUAL`.

6 Dashboard Subsystem (DBS)

L'interfaccia DBS rappresenta il pannello di gestione e visualizzazione dello stato del sistema.

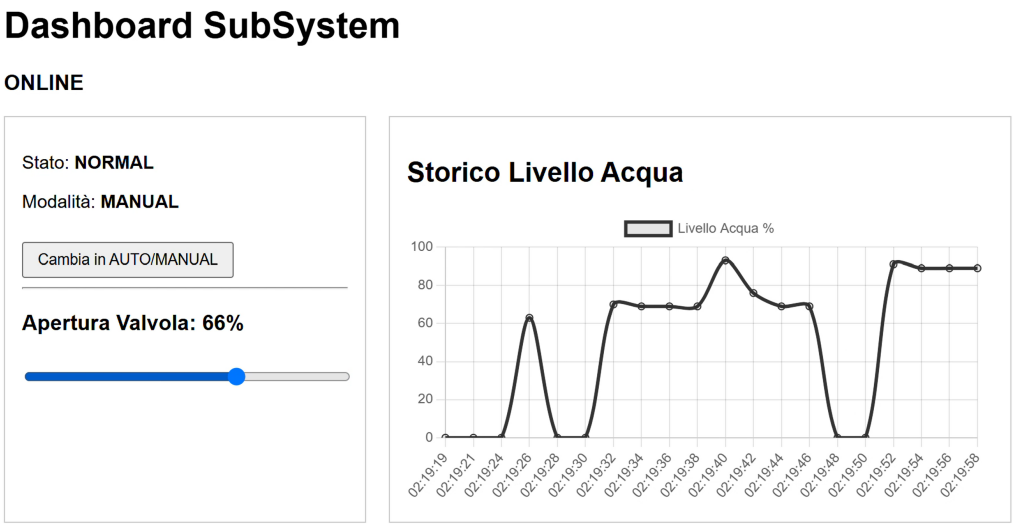


Figura 4: Dashboard DBS.

Implementata come singola pagina web, l'interfaccia svolge tre funzioni:

- **Monitoraggio dello stato della connessione e del livello dell'acqua:** Mostra lo stato di connessione del sistema, visualizza la percentuale di apertura della valvola e traccia l'andamento del livello dell'acqua tramite un grafico.
- **Selezione della modalità:** Consente all'operatore di alternare con un pulsante la gestione automatica e quella manuale. Passando in modalità manuale si abilita uno slider per impostare direttamente l'apertura della valvola.
- **Sincronizzazione della posizione della valvola:** Mantiene la schermata costantemente sincronizzata con l'hardware. Se l'operatore modifica l'apertura della valvola con il potenziometro, la pagina web aggiorna automaticamente la posizione del cursore a schermo per mostrare il valore corrente.

7 Video Dimostrativo

Il video dimostrativo è disponibile al seguente link: assignment-03.mp4.